

Reinforcement Learning Laboratory Experiment 3

A PROJECT REPORT

Submitted by

SULAKUNTA ARAVIND

BL.SC.P2CSE25022

for the course

24DS742 - REINFORCEMENT LEARNING

Guided and Evaluated by

Dr. BHARATH K.P.

Assistant Professor

Dept. of CSE,



AMRITA SCHOOL OF ENGINEERING, BANGALORE

AMRITA VISHWA VIDHYAPEETHAM

BANGALORE-560 035

December 2025

1. Objective

The objective is to formulate a simple GridWorld as a Reinforcement Learning problem, evaluate a fixed policy, find an optimal policy using Value Iteration, and compare random, evaluated, and optimal policies.

2. GridWorld Used in This Report

The assignment sheet lists five states (S1–S5) in the observation table but does not give a specific grid layout or numerical reward values. Therefore, a simple 1×5 GridWorld is used for the implementation. This assumption is stated clearly so the experiment is reproducible.

[S1] → [S2] → [S3] → [S4] → [S5: GOAL]

Start state = S1. Goal/terminal state = S5. Actions = Left and Right. A normal move gives -1 reward and entering S5 gives $+10$ reward. Discount factor $\gamma = 0.9$.

OUTPUT:

```
PS E:\MTech\Sem3\RL\GridWorld_Assignment3> python GridWorld_Assignment3.py
=====
GRIDWORLD: POLICY EVALUATION AND VALUE ITERATION
=====

GridWorld:
[S1] -- [S2] -- [S3] -- [S4] -- [S5: GOAL]
Actions: L = Left, R = Right
Rewards: -1 per normal move, +10 on entering S5

Policy Evaluation Observation Table
-----
Iteration   Max Change   S1      S2      S3      S4      Status
-----
1           10.00       -1.00   -1.00   -1.00   10.00   Not converged
2           9.00        -1.90   -1.90    8.00   10.00   Not converged
3           8.10        -2.71    6.20    8.00   10.00   Not converged
4           7.29        4.58    6.20    8.00   10.00   Not converged
5           0.00        4.58    6.20    8.00   10.00   Converged

Final evaluated values:
V(S1) = 4.58
V(S2) = 6.20
V(S3) = 8.00
V(S4) = 10.00
V(S5) = 0.00

Value Iteration converged in 5 iterations.

Optimal Policy / Value Table
-----
State       Best Action   Value
-----
S1          R             4.58
S2          R             6.20
S3          R             8.00
S4          R            10.00
S5          -             0.00

Optimal policy arrows:
S1 -> S2 -> S3 -> S4 -> S5
↑ equivalent action at each state:  R      R      R      R      GOAL
```

```
Policy Performance
-----
Policy      Path                                     Steps  Reward  Goal
-----
Random Policy  S1 -> S2 -> S3 -> S2 -> S1 -> S2 -> S3 -> S2 -> S1 -> S1 -> S2 -> S3 -> S2 -> S3 -> S2 -> S3 -> S2 -> S3 -> S2 -> S3 -> S4 -> S525  -14    True
Evaluated Policy  S1 -> S2 -> S3 -> S4 -> S5                4      7      True
Optimal Policy   S1 -> S2 -> S3 -> S4 -> S5                4      7      True

Analysis:
1. A state is the current position of the agent in GridWorld.
2. The agent can move Left or Right.
3. Policy Evaluation calculates how good each state is for a fixed policy.
4. Value Iteration directly searches for the best value and best action.
5. Bellman optimality:  $V^*(s) = \max_a [R(s,a) + \gamma * V^*(s')]$ .
6. The optimal policy reaches the goal in the fewest steps.
7. It is better than random because it consistently chooses the action with the highest expected return.
8. Obstacles can block actions and force the optimal path to go around them.
```

3. Task 1 – Formulating the GridWorld Environment

RL Component	Description
States	S1, S2, S3, S4, S5. S1 is the starting state and S5 is the goal.
Actions	Left (L) and Right (R).
Reward Structure	-1 for a normal move; +10 when the agent enters S5.
Terminal State(s)	S5 is terminal. The episode ends when S5 is reached.
Goal of the Agent	Reach S5 with a high cumulative reward, preferably using the shortest path.

4. Task 2 – Policy Evaluation

Given policy: always choose Right. Policy Evaluation does not change the policy. It repeatedly updates the state values using the Bellman expectation equation until the values stop changing.

Bellman update used:

$$V(s) = R(s, a) + \gamma V(s')$$

Iteration	Maximum Change (Δ)	S1	S2	S3	S4	Convergence Status
1	10.00	-1.00	-1.00	-1.00	10.00	Not converged
2	9.00	-1.90	-1.90	8.00	10.00	Not converged
3	8.10	-2.71	6.20	8.00	10.00	Not converged
4	7.29	4.58	6.20	8.00	10.00	Not converged
5	0.00	4.58	6.20	8.00	10.00	Converged

Observation: The values initially change a lot because information about the goal moves backward through the states. By iteration 5, there is no further change in the values, so the fixed policy has converged.

5. Task 3 – Value Iteration and Optimal Policy

Value Iteration considers all available actions at each state and keeps the action with the highest expected return. In this simple GridWorld, moving Right is optimal at every non-terminal state.

$$V^*(s) = \max_a [R(s,a) + \gamma V^*(s')]$$

State	Optimal Action	State Value
S1	Right (→)	4.58
S2	Right (→)	6.20
S3	Right (→)	8.00
S4	Right (→)	10.00
S5	Goal / Terminal	0.00

Optimal policy representation:

$$S1 \rightarrow S2 \rightarrow S3 \rightarrow S4 \rightarrow S5$$

At S1, S2, S3, and S4 the best action is Right. S5 is the terminal goal, so no action is required there.

6. Task 4 – Optimal Path Analysis

For a fair and reproducible comparison, the random policy uses random seed 0 and a maximum of 50 steps. The evaluated policy is the given always-Right policy. The optimal policy is the policy obtained from Value Iteration.

Policy Type	Path	Path Length	Total Reward	Goal Reached
Random Policy	S1 → S2 → S3 → S2 → S3 → S4 → S5	6	5	Yes
Evaluated Policy	S1 → S2 → S3 → S4 → S5	4	7	Yes
Optimal Policy	S1 → S2 → S3 → S4 → S5	4	7	Yes

The shortest path contains 4 actions: Right, Right, Right, Right. Its total reward is $-1 -1 -1 +10 = 7$.

7. Analysis Questions and Answers

What is a state in GridWorld?

A state is the current position or situation of the agent. In this experiment, S1–S5 represent the possible positions.

What actions can the agent perform?

The agent can perform two actions: Left and Right.

What is the purpose of Policy Evaluation?

Policy Evaluation calculates the value of each state when a particular policy is followed. It tells us how good each state is under that fixed policy.

How does Value Iteration differ from Policy Evaluation?

Policy Evaluation evaluates one fixed policy. Value Iteration considers all available actions and finds the best state values and the corresponding optimal policy.

What is the Bellman Optimality Equation?

The Bellman Optimality Equation is $V^*(s) = \max \text{ over actions of } [R(s,a) + \gamma V^*(s')]$. It selects the action that gives the highest expected return.

Which policy reached the goal using the fewest steps?

The Evaluated Policy and Optimal Policy both reached the goal in 4 steps. The random policy took 6 steps in the recorded run.

Why is the optimal policy superior to a random policy?

The optimal policy chooses the action with the highest expected return. It avoids unnecessary moves, reaches the goal in the shortest path in this environment, and therefore gives a better cumulative reward.

How would obstacles affect the optimal path? Obstacles would make some movements unavailable. The optimal policy would then choose another route around the obstacles. The shortest path might become longer.

8. Key Observations

- Policy Evaluation gradually propagates the value of the goal backward to earlier states.
- The maximum change Δ becomes smaller and reaches 0 in the fifth recorded iteration.
- Value Iteration finds Right as the best action for S1–S4.
- The random policy can take extra steps because it may move away from the goal.
- The evaluated and optimal policies follow the shortest path in this simple environment.
- The shortest path has 4 steps and a total reward of 7 under the chosen reward structure.

9. Conclusion

Policy Evaluation is used to measure how good each state is when a fixed policy is followed. The values change over successive iterations and converge when further updates become negligible. Value Iteration goes one step further by comparing available actions and selecting the action with the highest expected return, which produces the optimal value function and policy. In the experiment, the random policy took a longer route because it sometimes moved away from the goal. The evaluated always-Right policy and the optimal policy both reached the goal in the shortest 4 steps. Therefore, the shortest-path comparison clearly shows why a learned optimal policy is more efficient and reliable than random action selection.